Dynamic Programming Roadmap for Coding Interviews

Java-focused guide: recursion, memoization, tabulation, categories, and practice plan

Purpose: learn DP by problem pattern and state design—not by memorizing isolated solutions.

1. The four approaches

Recursion: Break a problem into smaller problems. Without caching, overlapping subproblems may be recomputed.

Memoization (top-down DP): Use recursion and cache each computed state. Return stored results when a state is requested again.

Tabulation (bottom-up DP): Compute smaller states first, usually with loops, and build toward the required answer.

Space-optimized DP: Keep only prior states needed by the recurrence instead of the entire table.

2. Complexity comparison

Plain recursion: Typically O(2^n) time and O(n) recursion-stack space for Fibonacci / Climbing Stairs.

Memoization: O(n) time and O(n) space for cache plus recursion stack.

Tabulation: O(n) time and O(n) space for the table.

Space optimized: O(n) time and O(1) auxiliary space when only the previous two states are needed.

3. DP problem categories and practice list

Basic 1D DP — dp[i]

LeetCode 509 Fibonacci Number (Easy); 70 Climbing Stairs (Easy); 746 Min Cost Climbing Stairs (Easy); 1137 N-th Tribonacci Number (Easy).

Take / Skip DP

198 House Robber (Medium); 213 House Robber II (Medium); 740 Delete and Earn (Medium); 2140 Solving Questions With Brainpower (Medium).

Grid / 2D DP — dp[row][col]

62 Unique Paths (Medium); 63 Unique Paths II (Medium); 64 Minimum Path Sum (Medium); 120 Triangle (Medium); 931 Minimum Falling Path Sum (Medium).

0/1 Knapsack

416 Partition Equal Subset Sum (Medium); 494 Target Sum (Medium); 1049 Last Stone Weight II (Medium); classic 0/1 Knapsack exercise.

Unbounded Knapsack / Coin Change

322 Coin Change (Medium); 518 Coin Change II (Medium); 279 Perfect Squares (Medium); 1449 Form Largest Integer With Digits That Add up to Target (Hard).

String DP / Two Sequences

1143 Longest Common Subsequence (Medium); 72 Edit Distance (Medium); 583 Delete Operation for Two Strings (Medium); 97 Interleaving String (Medium); 115 Distinct Subsequences (Hard).

Subsequence / LIS

300 Longest Increasing Subsequence (Medium); 673 Number of Longest Increasing Subsequence (Medium); 368 Largest Divisible Subset (Medium); 1027 Longest Arithmetic Subsequence (Medium).

Stock / State-Machine DP

121 Best Time to Buy and Sell Stock (Easy); 122 Stock II (Medium); 309 Stock with Cooldown (Medium); 714 Stock with Transaction Fee (Medium); 123 Stock III (Hard).

Interval / Partition DP

5 Longest Palindromic Substring (Medium); 516 Longest Palindromic Subsequence (Medium); 1039 Minimum Score Triangulation of Polygon (Medium); 312 Burst Balloons (Hard); 1547 Minimum Cost to Cut a Stick (Hard).

Tree DP

337 House Robber III (Medium); 124 Binary Tree Maximum Path Sum (Hard); 968 Binary Tree Cameras (Hard).

Graph DP / DAG

329 Longest Increasing Path in a Matrix (Hard); 787 Cheapest Flights Within K Stops (Medium); 2050 Parallel Courses III (Hard).

Bitmask DP

847 Shortest Path Visiting All Nodes (Hard); 1125 Smallest Sufficient Team (Hard); 943 Find the Shortest Superstring (Hard).

Digit DP / Advanced Counting

233 Number of Digit One (Hard); 902 Numbers At Most N Given Digit Set (Hard); 1012 Numbers With Repeated Digits (Hard).

4. How to solve a DP problem

1 — Define the state: Explain what dp[i], dp[i][j], or dp[index][capacity] means.

2 — Derive the recurrence: Identify the choices or prior states that determine the current answer.

3 — Set base cases: Define answers for the smallest or terminal states.

4 — Add memoization: Check the cache before calculating a recursive state.

5 — Choose tabulation order: Calculate dependencies before the state that needs them.

6 — Optimize space: Only after correctness, remove table entries that are no longer needed.

5. Category-to-state reference

Category	Typical state / evaluation order
Basic 1D / take-skip	dp[i] or solve(i); iterate in dependency order.
Grid	dp[row][col] or solve(r,c); process cells in an order consistent with allowed moves.
0/1 knapsack	dp[item][capacity]; each item once. A 1D version commonly iterates capacity downward.
Unbounded knapsack	dp[item][amount]; reuse allowed. A 1D version commonly iterates amount upward.
String DP	dp[i][j] for prefixes of two strings; fill smaller prefixes before dependent larger ones.
LIS	solve(i, previous) or dp[i]; O(n^2) DP or O(n log n) tails optimization.
Stock	dp[day][state] for holding/cash/transactions.
Interval	dp[left][right]; usually short intervals before long intervals.
Tree	solve(node,state); usually postorder, children before parent.
DAG	dp[node] or dp[node][state]; topological order is often useful.
Bitmask	dp[mask] or dp[mask][last]; build answers over subsets.
Digit DP	dp[position][tight][other state]; process digits left to right.

6. Worked example — House Robber

Problem: Maximize the amount robbed without choosing adjacent houses. Example [2, 7, 9, 3, 1] gives 12.

State: solve(i) is the best amount from index i onward.

Recurrence: solve(i) = max(nums[i] + solve(i+2), solve(i+1)); base case i >= n returns 0.

Plain recursion: int solve(int i, int[] nums) { if (i >= nums.length) return 0; return Math.max(nums[i] + solve(i+2, nums), solve(i+1, nums)); }

Memoization: Cache solve(i) in memo[i]. Each index is solved once, reducing time from exponential to O(n).

Tabulation: Iterate i from n-1 down to 0: dp[i] = max(nums[i] + dp[i+2], dp[i+1]); return dp[0].

Space optimization: Only dp[i+1] and dp[i+2] are needed, so two rolling variables give O(1) auxiliary space.

7. Identify a likely DP category

Choices at each position: Consider 1D DP, take/skip, knapsack, or state-machine DP.

Grid movement: Consider grid DP.

Two strings or sequences: Consider 2D sequence/string DP.

Subsequence constraints: Consider LIS-family DP.

Split a range / choose last operation: Consider interval or partition DP.

Combine child subtree results: Consider tree DP.

DAG dependencies: Consider DAG DP.

Subset decisions over a small set: Consider bitmask DP.

Count numbers under digit rules: Consider digit DP.

Caution: Minimum, maximum, count, or possible are clues, not proof. Look for reusable subproblems and a manageable state.

8. Recommended study order

Phase 1 — Foundations: Fibonacci → Climbing Stairs → Min Cost Climbing Stairs.

Phase 2 — Decision DP: House Robber → House Robber II.

Phase 3 — Grid DP: Unique Paths → Minimum Path Sum.

Phase 4 — Knapsack: Partition Equal Subset Sum → Coin Change → Coin Change II.

Phase 5 — String DP: Longest Common Subsequence → Edit Distance.

Phase 6 — Subsequences: Longest Increasing Subsequence.

Phase 7 — State machines: Stock problems.

Phase 8 — Advanced: Interval DP → Tree DP → DAG DP → Bitmask DP → Digit DP.

9. Interview routine

For every core problem: Define state; derive recurrence; state base cases; code recursion; add memoization; convert to tabulation; optimize space only when valid; state time and space complexity.

Key warning: Not every recursive/backtracking problem benefits from DP. Without meaningful overlapping subproblems, caching may not help. Some DP problems are naturally iterative.